

AI Root Cause Analysis Platform

Product Requirements / Business Requirements Document — v3

Expanded with Data Profiling, Schema Validation, and KPI Selection requirements

1. Product Vision

Build an AI-powered Root Cause Analysis platform that automatically detects unusual business behavior, investigates possible causes across multiple dimensions, and explains findings in business language.

2. Business Problem

Companies have large amounts of data. When a KPI changes, a business user normally has to ask a Data Analyst or Data Scientist to extract data using SQL, create charts, investigate dimensions, and identify possible causes. This process is time-consuming. The platform aims to reduce this manual investigation effort while keeping calculations and evidence traceable.

3. Example Output

Sales decreased by 14.8% in July. The primary driver was a 31% decrease in Product A sales in the Cairo region, contributing approximately 9.2 percentage points to the overall decline.

4. Example User Journey

Step 1: Select KPI = Revenue, Period = July 2026, Comparison = June 2026.

Step 2: Revenue \$10.2M → \$8.7M, Change -14.8%, Severity HIGH.

Step 3: AI investigates time-series anomaly, region, product, customer, contribution, and statistical validation.

Step 4: Primary root cause = Cairo → Product A → Enterprise; Contribution 58%; Revenue impact -\$700K; Confidence 92%.

Step 5: AI explains the result in business language.

5. First Phase — Data Foundation

Required capabilities:

- CSV upload
- Excel upload
- SQL Server data source
- Data profiling
- Schema validation
- KPI selection
- Dataset metadata management
- SQL editor for SQL Server
- Saving query output as an internal dataset

6. CSV Upload Requirements

Users can upload CSV files up to 200 MB. The upload flow validates the file before analysis. The physical storage key must use a UUID rather than the original filename to prevent collisions and avoid user-controlled filesystem paths. The original filename is retained as metadata. Uploads should use bounded streaming in 1 MB chunks. Storage is accessed

through an abstraction so the backend can later move from local storage to object storage.

7. Dataset Metadata

Minimum metadata:

- filename
- size
- format
- company_id
- user_id

Recommended production fields:

- dataset_id
- storage_key
- upload_status
- checksum
- created_at
- updated_at
- row_count
- schema_version

8. SQL Server Data Source

The first phase includes a SQL editor for authorized users to connect to SQL Server using server/host, database name, username, password, and SQL query. Users can execute a query, inspect the result, and save the query output as an internal dataset. Credentials must not be stored in plaintext in logs or dataset metadata.

9. Data Profiling

Data profiling answers: “**What is in this dataset?**”

After upload or after a SQL query is saved as a dataset, the backend automatically generates a profile.

Dataset-level profile:

- row count
- column count
- file size
- duplicate row count
- missing-cell percentage
- dataset status

Numeric columns:

- data type
- missing percentage
- unique count

- min / max
- mean / median
- standard deviation
- percentiles
- outlier indicators

Categorical columns:

- unique count
- missing percentage
- most frequent values
- value frequencies

Datetime columns:

- minimum date
- maximum date
- number of periods
- missing dates
- detected frequency where possible

The profile should be available through an API and a frontend Data Profile screen.

10. Schema Validation

Schema validation answers: “**Is this dataset structurally usable for analysis?**”

The validator checks expected or inferred data types, required fields for the selected analysis, convertible values, missing critical fields, and invalid values.

Example:

- revenue detected as string
- possible numeric conversion detected
- conversion confidence is high
- invalid values are reported

Validation must produce one of three states:

PASS — dataset is valid for analysis.

WARNING — dataset can be analyzed but has quality issues.

BLOCKED — dataset cannot safely be used for RCA.

The validator should attempt safe type inference/conversion where appropriate rather than rejecting every type mismatch.

11. KPI Detection and KPI Selection

KPI selection should be connected to profiling rather than being an isolated manual dropdown.

Data profiling identifies:

- numeric measure candidates
- datetime candidates
- categorical dimension candidates

Example dataset:

revenue, cost, profit, quantity, date, region, product, customer

The system can recommend Revenue, Profit, Quantity as KPI candidates; Date as the time dimension; and Region, Product, Customer as analysis dimensions.

The user then configures the analysis:

- KPI
- aggregation (SUM, AVG, COUNT, etc.)
- time dimension
- comparison period
- analysis dimensions

Example KPI definition:

```
{
  "name": "Revenue",
  "column": "revenue",
  "aggregation": "SUM",
  "time_column": "date",
  "dimensions": ["region", "product", "customer_segment"],
  "comparison": "previous_period"
}
```

The RCA engine should receive this normalized KPI definition rather than depending on whether the source was CSV, Excel, or SQL Server.

12. How the Three Features Work Together

These are three connected capabilities, not three isolated screens:

Upload / Connect Dataset

↓

Schema Validation

↓

Data Profiling

↓

KPI Detection

↓

User KPI Selection

↓

KPI Definition



RCA Engine

Schema Validation asks whether the dataset is structurally usable.

Data Profiling explains what the dataset contains.

KPI Detection identifies possible metrics, time fields, and dimensions.

KPI Selection defines exactly what the RCA engine should investigate.

This separation makes the system easier to extend and allows the RCA engine to work with any supported data source through the same analytical contract.

13. Recommended Frontend Flow

Dataset page: show dataset size, rows, columns, quality status, and a View Profile action.

Data Profile: show Overview, Columns, Quality, and Statistics tabs.

KPI Setup:

- KPI selector with recommended candidates
- aggregation selector
- time-dimension selector
- comparison selector
- analysis-dimension multi-select
- Start Analysis action

Once configured, the dataset becomes **Analysis Ready** and can be passed to the RCA engine.

14. Root Cause Analysis — Future Phase

The RCA engine should support:

- KPI anomaly detection
- period-over-period comparison
- time-series analysis
- dimension drill-down
- contribution analysis
- statistical validation
- root-cause ranking
- RCA tree generation
- evidence-backed AI explanations

The LLM should interpret validated analytical results rather than inventing statistical calculations.

15. Proposed Repository Structure

The project is split into two primary application folders — backend and frontend — with Docker and repository-level configuration files outside them.

```
ai-root-cause-analysis/
├── backend/
│   ├── app/
│   │   ├── api/
│   │   │   ├── routes/
│   │   │   │   ├── auth.py
│   │   │   │   ├── datasets.py
│   │   │   │   ├── uploads.py
│   │   │   │   ├── sql_connections.py
│   │   │   │   ├── sql_editor.py
│   │   │   │   ├── rca.py
│   │   │   │   └── router.py
│   │   ├── core/
│   │   │   ├── config.py
│   │   │   ├── security.py
│   │   │   ├── logging.py
│   │   │   ├── exceptions.py
│   │   ├── db/
│   │   │   ├── models/
│   │   │   ├── repositories/
│   │   │   ├── session.py
│   │   │   ├── migrations/
│   │   │   ├── schemas/
│   │   │   │   ├── auth.py
│   │   │   │   ├── dataset.py
│   │   │   │   ├── upload.py
│   │   │   │   ├── sql.py
│   │   │   ├── services/
│   │   │   │   ├── dataset_service.py
│   │   │   │   ├── profiling_service.py
│   │   │   │   ├── sql_service.py
│   │   │   │   └── storage_service.py
│   │   ├── storage/
│   │   │   ├── base.py
│   │   │   ├── local.py
│   │   │   ├── connectors/
│   │   │   │   └── sqlserver.py
│   │   ├── rca/
│   │   │   ├── anomaly_detection.py
│   │   │   ├── dimension_analysis.py
│   │   │   ├── contribution.py
│   │   │   ├── statistical_validation.py
│   │   │   └── ranking.py
│   │   └── main.py
│   ├── tests/
│   │   ├── unit/
│   │   └── integration/
│   ├── requirements.txt
│   ├── Dockerfile
│   └── .dockerignore
├── frontend/
│   ├── src/
│   │   ├── app/
│   │   │   ├── components/
│   │   │   ├── features/
│   │   │   │   ├── datasets/
│   │   │   │   └── sql-editor/
│   │   │   └── rca/
│   │   ├── lib/
│   │   ├── hooks/
│   │   ├── types/
│   │   ├── public/
│   │   ├── package.json
│   │   ├── Dockerfile
│   │   └── .dockerignore
│   ├── docker/
│   │   ├── nginx/
│   │   │   └── nginx.conf
│   │   ├── docker-compose.yml
│   │   ├── .env.example
│   │   ├── .gitignore
│   │   ├── README.md
│   │   └── LICENSE
```

16. Backend Responsibilities

Module	Responsibility
API	FastAPI endpoints, validation, authentication and authorization.
Storage	Streaming file writes through a storage abstraction and UUID keys.
Dataset Service	Dataset registration, metadata, lifecycle and ownership.
Profiling	Schema, types, nulls, duplicates, statistics and data-quality checks.
SQL Connector	Secure SQL Server connection and query execution.
SQL Editor Service	Execute queries, validate results, preview and save output as datasets.
RCA Engine	Anomaly, contribution, dimension and statistical analysis.
Database	PostgreSQL persistence for users, companies, datasets and metadata.

17. Frontend Responsibilities

The frontend should provide:

- Authentication and company-aware access
- Dataset upload screen with progress and validation state
- Dataset list and metadata view
- Data Profile screen
- KPI Setup screen
- SQL Server connection form
- SQL editor with execute and save-output actions
- RCA investigation dashboard
- RCA tree and evidence visualization
- Investigation history

18. Docker & Environment Files

docker-compose.yml should orchestrate backend, frontend, PostgreSQL, optional Redis, and optional Nginx.

Dockerfiles should exist independently under backend and frontend. **.env.example** documents required environment variables without secrets. **.gitignore** excludes secrets, virtual environments, build output, local uploads, caches, IDE files, logs, and generated artifacts.

19. Suggested .gitignore Scope

```
# Python
__pycache__/
*.py[cod]
.venv/
venv/

# Environment / secrets
.env
.env.*
!.env.example

# Local data
uploads/
storage/
*.db

# Tests / tooling
.pytest_cache/
.mypy_cache/
.coverage
htmlcov/

# Frontend
node_modules/
.next/
dist/
build/

# Logs
*.log
logs/

# IDE / OS
.vscode/
.idea/
.DS_Store
Thumbs.db
```

20. Engineering Principles

1. Never load a 200 MB upload fully into memory just to store it.
2. Never use the original filename as the physical storage path.
3. Keep storage behind an abstraction so local storage can later be replaced by object storage.
4. Separate API, service, repository, connector and analytical layers.
5. Keep LLM reasoning separate from deterministic data analysis.
6. Every RCA claim should be traceable to an analytical result.
7. Keep secrets out of Git, logs and normal metadata.
8. Build the data foundation before the AI investigation layer.

21. MVP Definition of Done

The first milestone is complete when an authenticated user can upload a valid CSV up to 200 MB, the backend streams and stores it using a UUID key, metadata is persisted in PostgreSQL, the dataset can be profiled, and an authorized user can connect to SQL Server, execute a query, preview the result, and save the result as a dataset. The dataset should then be capable of passing schema validation, exposing profiling results, allowing KPI selection, and producing an Analysis Ready KPI definition for the RCA engine.